מספר התלמיד הנבחן
רשום את כל תשע הספרות

הדבק כאן את מדבקת הנבחן

האוניברסיטה הפתוחה

א' באב תשע"ז

24
ביולי 2017

סמסטר 2017ב

מס' שאלון - 517

מס' מועד 86

20905 / 4

שאלון בחינת גמר

20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה **10** עמודים

מבנה הבחינה:

בבחינה ארבע שאלות.

עליכם לענות על כל השאלות.

משקל השאלות מפורט בגוף השאלות.

שימו לב:

את תשובותיכם רשמו אך ורק במחברת הבחינה.

כתבו בכתב יד ברור ומסודר.

חומר עזר:
ספר הקורס: essentials of programming languages
מדריך הלמידה. מותרות הערות בכתב יד, ע"ג הספרים.
אין להכניס חומר מודפס או כל חומר אחר מכל סוג שהוא.

בהצלחה !!!

אינכם חייבים
להחזיר את השאלון לאוניברסיטה הפתוחה

שאלה 1 (25 נקודות)

שאלה זו עוסקת בשפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81.
כזכור, שפת "וישגור" מאפשרת הגדרת פרוצדורות בעלות פרמטר יחיד.
להזכירכם, **משתנה חופשי** בפרוצדורה הוא משתנה המוזכר בגוף הפרוצדורה ואינו אחד מהפרמטרים של הפרוצדורה.
להלן דוגמא להבהרה,

```
let x=9
in
  let f= proc (y)  -(x,y)
  in
   (f 4)
```

בדוגמא זו, המשתנה x המוזכר בגוף הפרוצדורה f הוא משתנה חופשי, ואילו y הפרמטר של הפרוצדורה אינו משתנה חופשי.

האם שינוי שמו של הפרמטר של פרוצדורה (כלשהי) בשפת PROC לשם אחר, שעדיין שונה משמות המשתנים החופשיים בפרוצדורה, עלול לגרום לקבלת תוצאת ריצה שונה?
אם התשובה היא לא, הסבר מדוע?
אם התשובה היא כן, הבא דוגמא לתוכנית בשפת PROC הממחישה קבלת תוצאת ריצה שונה.

בתשובתך, התייחס למקרים הבאים:

א) (12 נקודות)

מפרש בו הפרוצדורות מחושבות על פי כריכה סטטית (static binding)

ב) (13 נקודות)

מפרש בו הפרוצדורות מחושבות על פי כריכה דינמית (dynamic binding)

שאלה 2 (25 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81. שפה זו מאפשרת הגדרה והפעלה של פרוצדורות לא רקורסיביות.

ברצוננו להרחיב את השפה עם ביטוי בוליאני שתחבירו נתון על-ידי הדקדוק הבא:

Expression ::= **check** ***Expression*** {**and** ***Expression***}* **end**

```
check-exp (bexp bexps)
```

ביטוי **check** מאפשר שרשור של כמה ביטויים בוליאניים עם פעולת **and** ביניהם. המשמעות של ביטוי **check** היא בוליאנית, וערכו הוא **true** אם ורק אם ערכו של כל אחד מהביטויים הבוליאניים ששורשרו על-ידי **and** הוא **true**. אחרת תוצאתו היא **false**. בחישוב של ביטוי **check** הביטויים הבוליאניים מחושבים משמאל לימין, והחישוב נעשה בצורת *short-circuit*, דהיינו, אם ערכו של אחד מהביטויים הבוליאניים הוא **false**, החישוב נפסק וערכו של כל הביטוי **check** הוא **false**. על מנת לסייע למנתח התחבירי (parser) לזהות ביטוי בוליאני, הביטוי מתחיל במילה **check** ומסתיים במילה **end**.

בתחביר מופשט, הביטוי נקרא **check-exp** והוא מורכב מ- **bexp** המציין את הביטוי הבוליאני הראשון ומ- **bexps** שהיא רשימה של יתר הביטויים הבוליאניים.

להלן דוגמאות מספר שימחישו את השימוש בביטוי החדש:

check zero? (x) **end**
check zero? (x) **and** zero? (y) **end**
check zero? (x) **and** zero? (-(y,1)) **end**
check (proc (x) zero? (x) 7) **end**
if check zero? (x) **and** zero? (y) **end then** 3 **else** 4

א. (7 נקודות)

להלן נתונים ה-**scanner** וה-**parser** של שפת "**וישגור**". עליכם להשלים את החסר בריבוע המסומן.**<u>את התשובה כיתבו במחברת הבחינה</u>**.

ה-scanner:

```
(define the-lexical-spec
'((whitespace (whitespace) skip)
(comment ("%" (arbno (not #\newline))) skip)
  (identifier(letter (arbno (or letter digit "_" "-" "?")))
           symbol)
  (number (digit (arbno digit)) number)
(number ("-" digit (arbno digit)) number)
))
```

ה-parser:

```
(define the-grammar
    '((program (expression) a-program)

      (expression (number) const-exp)
      (expression
        ("-" "(" expression "," expression ")")
diff-exp)

      (expression
       ("zero?" "(" expression ")")
zero?-exp)

      (expression
       ("if" expression "then" expression "else" expression)
if-exp)

      (expression (identifier) var-exp)

      (expression
       ("let" identifier "=" expression "in" expression)
let-exp)

      (expression
       ("proc" "(" identifier ")" expression)
proc-exp)

      (expression
       ("(" expression expression ")")
call-exp)
```

```
(expression
        את התשובה יש לכתוב במחברת הבחינה
check-exp)))
```

ב. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת **"וישגור"** . ולאחריה פרוצדורת העזר apply-procedure כפי שמופיעה בספר הלימוד בעמוד 79. יש להשלים את החסר בפרוצדורה value-of. **<u>את התשובה כיתבו במחברת הבחינה</u>**.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וישגור".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

```
;;value-of : Exp x Env→ExpVal
(define value-of
(lambda (exp env)
(cases expression exp
(const-exp (num) (num-val num))
(var-exp (var) (apply-env env var))
(diff-exp (exp1 exp2)
  (let ((val1 (value-of exp1 env))
(val2 (value-of exp2 env)))
(let ((num1 (expval->num val1))
   (num2 (expval->num val2)))
(num-val(- num1 num2)))))
(zero?-exp (exp1)
(let ((val1 (value-of exp1 env)))
(let ((num1 (expval->num val1)))
(if (zero? num1)(bool-val #t)(bool-val #f)))))
(if-exp (exp1 exp2 exp3)
 (let ((val1 (value-of exp1 env)))
 (if (expval->bool val1)
(value-of exp2 env)
(value-of exp3 env))))
       (let-exp (var exp1 body)
 (let ((val1 (value-of exp1 env)))
 (value-of body(extend-env var val1 env))))

     (proc-exp (var body)
          (proc-val (procedure var body env)))

(call-exp (rator rand)
          (let ((proc (expval->proc (value-of rator env)))
                (arg (value-of rand env)))
            (apply-procedure proc arg)))
```

```
(check-exp  (_________________________________)


                 את התשובה יש לכתוב במחברת הבחינה
                                                  ))))
```

```
;; apply-procedure : Proc * ExpVal ->ExpVal
;; Page: 79
  (define apply-procedure
    (lambda (proc1 val)
      (cases proc proc1
        (procedure (var body saved-env)
          (value-of body (extend-env var val saved-env))))))
```

המשך הבחינה בעמוד הבא

שאלה 3 (25 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת **"וירמוז"**(IMPLICIT-REFS).

ברצוננו להרחיב שפה זו עם ביטוי חדש שיאפשר לבצע לולאת for.

ערכו של ביטוי זה יהיה סכום ערכי הביטויים שבוצעו במהלך מחזורי הלולאה.

להלן הדקדוק המגדיר את הביטוי החדש שברצוננו להוסיף:

Expression* ::=** for (identifier= ***Expression ; ***Expression***; ***Expression***) ***Expression***

for-exp (tmpid initvalue bool inc body)

לביטוי **for-exp** המרכיבים הבאים:

- **tmpid** – משתנה זמני שמוכר רק במסגרת פעולת הלולאה. כלומר אחרי הלולאה חוזרים לעבוד עם הסביבה שהייתה טרם ביצוע לולאת ה-for.
- **initvalue** – ערך התחלתי שיינתן למשתנה הזמני לפני ביצוע מחזורי הלולאה.
- **bool** – ביטוי בוליאני שכל עוד ערכו **false** הלולאה תימשך (בניגוד ל-for המוכר לכם משפות אחרות שבו הלולאה נמשכת כל עוד התנאי נכון), כאן התנאי משמש כתנאי עצירה ברגע שערכו true.
- **inc** – ביטוי חשבוני שערכו יתווסף לערכו של המשתנה הזמני בכל סוף מחזור של הלולאה.
- **body** – ביטוי המשמש כגוף הלולאה.

שלבי פעולת הלולאה דומים ללולאת for המוכרת לכם משפות אחרות, בתחילה המשתנה הזמני מקבל את הערך ההתחלתי, לאחר מכן נבדק תנאי הלולאה, אם ערכו true הלולאה נעצרת, ואם ערכו false הלולאה נמשכת ועוברים לבצע את גוף הלולאה, לאחר ביצוע גוף הלולאה מוסיפים למשתנה הזמני את ערכו של הביטוי **inc**, ושוב חוזרים לבדוק את ערכו של הביטוי הבוליאני כמו קודם, וכן הלאה.

בסיום הלולאה, ערכו של ביטוי ה- for יהיה שווה לסכום ערכי הביטויים שבוצעו בגוף הלולאה במהלך מחזורי הלולאה.

להלן דוגמא:

```
for(x=10; zero?(x); -2)
    -(x,-100)
```

בדוגמה זו, הלולאה מתחילה עם ערך של 10 ובכל מחזור הוא קטן ב-2 עד להגעה לאפס, מה שיגרום לעצירת הלולאה. כלומר הלולאה רצה עם ערכי x הבאים: 10,8,6,4,2, בכל מחזור לולאה מבוצע גוף הלולאה, שהוא הביטוי x+100, ולכן ערכו של גוף הלולאה במחזורי הלולאה השונים הוא: 110,108,106,104,102 בהתאמה לערכי x השונים.

ערכו של ביטוי ה-for בסיום הלולאה יהיה סכום הערכים של גוף הלולאה בכל המחזורים שלה, דהיינו, 110+108+106+104+102 השווה ל- (num-val 530).

א. (7 נקודות)

להלן נתונים ה-**scanner** וה-**parser** של שפת "**וירמוז**". עליכם להשלים את החסר בריבוע המסומן.**<u>את התשובה כיתבו במחברת הבחינה</u>**.

ה-scanner:

```
(define the-lexical-spec
'((whitespace (whitespace) skip)
(comment ("%" (arbno (not #\newline))) skip)
  (identifier(letter (arbno (or letter digit "_" "-" "?")))
           symbol)
  (number (digit (arbno digit)) number)
(number ("-" digit (arbno digit)) number)
))
```

ה-parser:

```
(define the-grammar
'((program (expression) a-program)
  (expression (number) const-exp)
  (expression ("-" "(" expression "," expression ")")
    diff-exp)
  (expression ("zero?" "(" expression ")") zero?-exp)
  (expression
   ("if" expression "then" expression "else" expression)
     if-exp)
  (expression (identifier) var-exp)
  (expression ("let" identifier "=" expression "in" expression)
     let-exp)
  (expression ("proc" "(" identifier ")" expression)
     proc-exp)
  (expression ("(" expression expression ")") call-exp)
  (expression
        ("letrec"
          (arbno identifier "(" identifier ")" "=" expression)
           "in" expression)
letrec-exp) (expression ("set" identifier "=" expression)
    assign-exp)
(expression (
```

את התשובה כיתבו במחברת הבחינה

```
)
  for-exp)))
```

ב. (18 נקודות)

להלן הפרוצדורות *value-of* ו- *apply-procedure* של שפת **"וירמוז"**.

הסבירו במדויק מה הם השינויים והתוספות הנדרשים לשילוב הביטוי החדש בשפה זו.

<u>את התשובה כיתבו במחברת הבחינה</u>.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- המנשק לעבודה עם החסן (store) הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

```
;; value-of : Exp * Env ->ExpVal
;; Page: 118, 119
  (define value-of
    (lambda (exp env)
      (cases expression exp
        (const-exp (num) (num-val num))

       (var-exp (var) (deref (apply-env env var)))

        (diff-exp (exp1 exp2)
          (let ((val1 (value-of exp1 env))
                (val2 (value-of exp2 env)))
            (let ((num1 (expval->num val1))
                  (num2 (expval->num val2)))
              (num-val
                (- num1 num2)))))

        (zero?-exp (exp1)
          (let ((val1 (value-of exp1 env)))
            (let ((num1 (expval->num val1)))
              (if (zero? num1)
                (bool-val #t)
                (bool-val #f)))))

        (if-exp (exp1 exp2 exp3)
          (let ((val1 (value-of exp1 env)))
            (if (expval->bool val1)
              (value-of exp2 env)
              (value-of exp3 env))))

        (let-exp (var exp1 body)
          (let ((v1 (value-of exp1 env)))
            (value-of body
              (extend-env var (newref v1) env))))

        (proc-exp (var body)
          (proc-val (procedure var body env)))

        (call-exp (rator rand)
          (let ((proc (expval->proc (value-of rator env)))
                (arg (value-of rand env)))
            (apply-procedure proc arg)))
```

```
      (letrec-exp (p-names b-vars p-bodies letrec-body)
          (value-of letrec-body
            (extend-env-rec* p-names b-vars p-bodies env)))

        (assign-exp (var exp1)
          (begin
            (setref!
              (apply-env env var)
              (value-of exp1 env))
            (num-val 27))))))


(define apply-procedure
      (lambda (proc1 val)
         (cases proc proc1
            (procedure (var body saved-env)
               (value-of body
                  (extend-env var (newref val) saved-env))))))
```

שאלה 4 (25 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
let  y = 9
in
 let z = zero? (-(y,3))
 in
  proc (w)
   if z  then (w z) else z
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים).לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (10 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב- substitution ו- unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

ב ה צ ל ח ה !